# LabFlow Software Requirements Specification

| Document field | Value |
|---|---|
| Project | LabFlow — College Laboratory Equipment Management System |
| Document version | 0.7 |
| Status | Prepared for academic submission |
| Prepared by | Milan KB / SHR25CS176 (Sr No: 225763) |
| Institution | Sahrdaya College of Engineering and Technology, Kodakara |

## Revision history

| Version | Date | Author | Change |
|---|---|---|---|
| 0.7 | 2026-10-25 | Project author | Consolidated individual capstone specification for submission. Business rules and endpoint scope are unchanged from revision 0.6. Earlier revisions remain in repository history. |

## 1. Introduction

### 1.1 Purpose

This Software Requirements Specification (SRS) defines the MVP requirements and verification basis for LabFlow. It is intended for the individual student developer and the ICTAK assessor.

The document uses an IEEE-inspired structure and accompanies the separate final project report.

### 1.2 Problem statement

Laboratory equipment lending can become unreliable when availability, approvals, handovers, returns and damage are recorded in disconnected or informal records. The laboratory needs one demonstrable system that preserves each asset's current state and borrowing history while preventing unauthorized access and conflicting allocations.

### 1.3 Scope

LabFlow is a dedicated web application for college laboratory equipment management. It tracks each physical asset using a unique asset tag. Students can register, authenticate, browse equipment, request one asset at a time, review their request history and report damage for issued equipment. Lab administrators manage inventory, decide requests, record issue and return, handle maintenance and view operational summaries.

The MVP is not a production campus identity, finance, notification or repair-management system.

### 1.4 Objectives

1. Maintain an accurate catalog and current lifecycle state for individually tagged assets.
2. Provide students with a clear, accessible borrowing request and history flow.
3. Give administrators controlled inventory and circulation operations.
4. Enforce authentication, role permissions, ownership and input validation on the server.
5. Prevent duplicate allocation and partial lifecycle updates under concurrent actions.
6. Produce a responsive MERN application and evidence suitable for ICTAK assessment.

### 1.5 Stakeholders

| Stakeholder | Interest |
|---|---|
| Student borrower | Discover available equipment, request it and track only their own borrowing records. |
| Lab administrator | Maintain inventory and control every physical lending transition. |
| College laboratory | Preserve consistent asset availability and borrowing history. |
| Individual student developer | Build, test, document and demonstrate the capstone. |
| ICTAK/LMS reviewer | Assess requirements, implementation evidence, documentation and live core flows. |

### 1.6 Requirement status and priority

- **Approved baseline:** accepted project scope or business policy; this label is not a record of academic approval.
- **Provisional:** a target or parameter whose verification remains limited, as identified in this specification and the quality evidence.
- **TBD:** unresolved; no behavior may be inferred silently.
- **Must:** necessary for MVP acceptance.
- **Should:** important, but may be deferred only with an explicit reason.

Requirements describe expected behavior. Completed verification results and remaining limitations are recorded separately in the project report and quality documentation.

## 2. System overview

### 2.1 Product perspective

LabFlow has four logical capabilities:

| Capability | Responsibility | Depends on |
|---|---|---|
| Identity | Accounts, sessions, roles and ownership enforcement | — |
| Inventory | Asset records, catalog and condition state | Identity |
| Circulation | Requests, reservation, issue, return and damage | Identity, Inventory |
| Dashboard | Read-only operational summaries | All above |

### 2.2 MERN architecture

```
React application in browser
        |
        | HTTPS JSON requests
        v
Express / Node.js API
        |
        | validated Mongoose operations and transactions
        v
MongoDB replica set
```

- React must never connect directly to MongoDB.
- The API is the security and validation boundary.
- Frontend HTTP calls must use one API-client module.
- Shared borrowing transitions must be implemented once on the server.
- The project will use one repository with separate client and server areas.
- The demo implementation language is JavaScript/JSX.

### 2.3 Deployment assumptions

| Assumption | Status |
|---|---|
| Vercel serves the built React assets and an Express Function at /api, under one origin. Local Node startup remains supported. | Implemented and verified on the public demo on 23 October 2026 |
| Production uses Atlas Free with replica-set transaction support. | Selected provider; 32 isolated cloud acceptance tests passed on 24 October 2026 |
| Frontend and API use one origin to simplify cookie security and CORS. | Selected implementation baseline |
| HTTPS and provider-managed environment variables are required when deployed. | Approved |
| Only free services may be selected unless paid use is explicitly approved. | Approved |

| Assumption | Status |
|---|---|
| Vercel Hobby and Atlas Free (AWS Mumbai) are used at https://labflow-lemon.vercel.app; the API function region is bom1. See DEPLOYMENT.md for evidence and limits. | Demo deployment verified; no production availability guarantee |

The separate synthetic-demo Atlas project permits Vercel connections through an explicitly authorized 0.0.0.0/0 network rule. Database authentication, TLS and readWrite restricted to labflow remain mandatory. No public database credentials or real-person data are permitted. This synthetic-demo exception removes an IP restriction and is not a production baseline for real college use. The deployment runbook records its scope and operational limits.

## 3. User roles and permissions

Public registration creates only STUDENT accounts. Administrator provisioning is controlled outside public registration through npm run admin, reading ignored runtime secrets and refusing elevation of an existing account. See docs/SECURITY.md.

| Capability | Public | Student | Admin |
|---|---|---|---|
| Register as student / log in | Yes | — | — |
| Log out / read own session identity | — | Yes | Yes |
| Browse equipment catalog/details | — | Non-archived only | All, including archived history |
| Create borrowing request | — | Yes | No |
| Read borrowing requests | — | Own only | All, with filters |
| Cancel eligible request | — | Own only | Approved, unissued request with reason |
| Report damage | — | Own issued request | No |
| Create/update/maintain/ archive equipment | — | No | Yes |
| Approve/reject request | — | No | Yes |
| Record issue/return and inspect damage | — | No | Yes |
| View dashboard | — | No | Yes |
| Create an admin through public API | No | No | No |

Authorization must be enforced by the API for every protected request. Hiding a UI control is not authorization.

# 4. Functional requirements

F1–F11 are **Must** capabilities. Their acceptance criteria define the functional verification basis; results are recorded separately.

### F1 — Student registration and authentication

| Field | Specification |
|---|---|
| Actors | Visitor, Student, Admin |
| Requirement | The system shall register students and support login, session restoration and logout for valid users. |
| Acceptance criteria | (1) Valid student registration creates one account and returns a safe user representation. (2) Email is normalized; a duplicate returns 409 and creates nothing. (3) Missing, malformed or out-of-range input is rejected and persisted nowhere. (4) A supplied role or admin field cannot create/elevate an admin. (5) Valid login starts a new session; invalid credentials do not disclose which field failed. (6) Logout destroys the session. (7) Passwords and hashes are never returned or logged. |

Demo registration accepts only name, email and password; no student number or college-verification field is required. Password/session limits are defined under NFR-SEC and docs/SECURITY.md.

### F2 — Access control and ownership

| Field | Specification |
|---|---|
| Actors | Student, Admin |
| Requirement | The API shall enforce authentication, role authorization and record ownership. |
| Acceptance criteria | (1) Missing or expired authentication returns 401. (2) A student invoking an admin operation receives 403. (3) A student can list and read only their own requests. (4) An inaccessible request belonging to another student is not disclosed and returns 404. (5) Role and owner IDs come from server-controlled identity, never client claims. |

### F3 — Equipment discovery

| Field | Specification |
|---|---|
| Actors | Student, Admin |
| Requirement | Authenticated users shall browse equipment and identify current availability. |
| Acceptance criteria | (1) The catalog supports bounded search, category/status filters and pagination. (2) Default page size is 20 and maximum is 50. (3) Each result exposes asset tag, name, category, location and current state but no borrower data. (4) Detail lookup returns the selected asset or 404. (5) Loading, success, empty and retryable-error states are present. |

Student list/filter results exclude ARCHIVED assets, and direct student detail lookup for an archived asset returns 404. Admins retain archived inventory and history access; students retain their own request snapshots, not archived discovery access.

### F4 — Inventory management

| Field | Specification |
|---|---|
| Actors | Admin |
| Requirement | An administrator shall create, update, place in/out of maintenance and archive equipment. |
| Acceptance criteria | (1) Required fields and enums are validated server-side. (2) Asset tags are immutable, normalized and unique; duplicates return 409. (3) Updates require the expected version; stale updates return 409. (4) Maintenance/restoration records an admin note. (5) Delete semantics archive rather than erase. (6) Archive is rejected while an open request exists. (7) Historical request snapshots remain readable after archive. |

An **open (active) request** is PENDING, APPROVED or ISSUED. Manual entry into maintenance and archive require no open requests. Restoration from maintenance may retain pending requests but requires no allocation. Mandatory damaged-return maintenance is not subject to the manual-entry guard; see Section 6.3. PATCH actions and metadata allowlists are defined in Section 8.2.

### F5 — Borrowing request and history

| Field | Specification |
|---|---|
| Actors | Student |
| Requirement | A student shall select an available asset or describe Other equipment, and review the resulting history. |

| Field | Specification |
| --- | --- |
| Acceptance criteria | (1) Creation uses the authenticated student, valid asset or Other description, purpose, requested due date and stable request key. (2) A pending request does not reserve the asset. (3) The same student cannot have multiple active requests for the same asset. (4) Repeating the same request key and payload returns the original request; a changed payload conflicts. (5) History is paginated and shows state, decision reason where applicable and lifecycle timestamps. |

Registered-asset requests require AVAILABLE, non-archived equipment and a future requestedDueAt. Other equipment requests instead store a required 10–500 character requestedEquipment description and remain PENDING without an allocation until an admin assigns an available registered asset at approval. One unassigned open request per student is allowed; the existing uniqueness guard remains. Owner cancellation and admin rejection require no asset. Equipment/purpose dropdowns offer Other with required text; stored purpose remains 10–500 characters. Date rules and expired pending-request handling are defined in Section 6.3. Identical replay returns the existing request even if its original date has since expired; it does not create a new request or bypass approval checks.

### F6 — Request approval and rejection

| Field | Specification |
| --- | --- |
| Actors | Admin |
| Requirement | An administrator shall approve or reject a pending request. |
| Acceptance criteria | (1) Approval requires a valid due date and atomically changes the request to APPROVED and asset from AVAILABLE to RESERVED. (2) Rejection requires a non-empty reason and does not change asset state. (3) A decision on a non-pending request returns 409. (4) If two admins approve competing requests simultaneously, exactly one succeeds and the other receives 409. (5) No partial request/equipment update survives failure. |

For an unassigned Other request, approval requires a registered AVAILABLE equipmentId, no existing open request by that student for the selected asset, and an atomic snapshot/assignment/reservation. The original requested description is retained for history and idempotent replay. A registered-asset request cannot be reassigned to another asset.

At approval, both the original requestedDueAt and approved dueAt must still be future instants. The admin may choose a different dueAt, but it must be no later than seven days from the server-recorded approval instant. Expired pending requests cannot be approved; they remain pending until explicitly rejected or cancelled.

### F7 — Request cancellation

| Field | Specification |
|---|---|
| Actors | Student owner, Admin |
| Requirement | An eligible unissued request shall be cancellable without corrupting inventory. |
| Acceptance criteria | (1) A student can cancel their own PENDING or APPROVED request. (2) An admin can cancel an APPROVED, unissued request and must provide a reason. (3) Cancelling APPROVED atomically releases RESERVED equipment to AVAILABLE. (4) Repeated cancellation or cancellation after issue returns 409 and changes nothing. |

### F8 — Equipment issue and return

| Field | Specification |
|---|---|
| Actors | Admin |
| Requirement | An administrator shall record physical issue and return. |
| Acceptance criteria | (1) Issue requires matching APPROVED request and RESERVED equipment with a valid future due date. (2) Issue atomically changes both records to ISSUED and records time/actor. (3) Return requires matching issued records and a serviceable/damaged condition. (4) Serviceable return makes equipment AVAILABLE; damaged return makes it MAINTENANCE. (5) Damaged return requires an inspection note. (6) Repeated return returns 409 and cannot release or alter the asset twice. |

Issue cannot extend or replace dueAt; an expired approved reservation returns 409 and remains reserved until explicit cancellation. Every existing damage report must be resolved during return, including serviceable returns. A damaged return enters maintenance even when unrelated pending requests exist; those requests are retained without automatic rejection.

### F9 — Damage reporting and inspection

| Field | Specification |
|---|---|
| Actors | Student owner, Admin |
| Requirement | A student shall report damage for equipment currently issued to them, and the admin shall record inspection outcome during return. |

| Field | Specification |
|---|---|
| Acceptance criteria | (1) Only the owner of an ISSUED request can create the report. (2) The description is required and bounded. (3) At most one report exists per borrowing. (4) Admin request views identify unreviewed reports. (5) Return addresses an existing report and records inspection actor/time/result. (6) Return and report operations cannot interleave into an unreviewed report after completion. |

If a report exists, return requires a non-empty inspection note and an explicit inspection result matching the returned condition (SERVICEABLE or DAMAGED). The same transaction records reviewer/time/result and resolves the report. A serviceable decision must explain the reported issue; resolution records inspection, not proof of repair. No report is deleted or left unreviewed by a completed return.

### F10 — Administrator dashboard

| Field | Specification |
|---|---|
| Actors | Admin |
| Requirement | The system shall provide a read-only operational dashboard. |
| Acceptance criteria | (1) It shows equipment counts by state, pending requests, issued loans, derived overdue loans and unreviewed damage reports. (2) Counts match known database fixtures used for verification. (3) Empty and failure states are clear. (4) Student access returns 403. |

### F11 — Accessible responsive interaction

| Field | Specification |
|---|---|
| Actors | Visitor, Student, Admin |
| Requirement | Users shall complete core flows on small and large screens using pointer or keyboard. |
| Acceptance criteria | (1) Layouts remain usable at 320 CSS pixels and desktop width. (2) Every interactive control has an accessible name and visible focus. (3) Input errors are programmatically associated with their fields. (4) Status is not communicated by colour alone. (5) Submission controls prevent repeat activation while pending and outcomes are announced. (6) Semantic headings, landmarks, labels and appropriate image text are used. |

# 5. Nonfunctional requirements

## 5.1 Security and privacy

| ID | Priority / status | Requirement and measurable verification |
|---|---|---|
| NFR-SEC-01 | Must / Selected implementation baseline | Passwords shall use salted asynchronous scrypt with N=131072, r=8, p=1, 16-byte random salts and 64-byte derived keys. Passwords are 12–128 characters and at most 512 UTF-8 bytes, without trimming/truncation. No plaintext password or hash is returned or logged. At most two hash jobs run concurrently. |
| NFR-SEC-02 | Must / Selected implementation baseline | MongoDB-backed server sessions shall have an eight-hour absolute lifetime enforced independently of TTL cleanup. Production cookies are host-only __Host-labflow.sid, HttpOnly, Secure, SameSite=Lax and path /. Login regenerates; logout destroys. No browser token storage. |
| NFR-SEC-03 | Must / Approved | Every protected endpoint shall pass tests for valid role, missing authentication and wrong role; owner-scoped endpoints shall also pass cross-user denial tests. |
| NFR-SEC-04 | Must / Approved | Body, query, path, relevant headers and environment configuration shall be allowlist-validated. NoSQL operators and unapproved fields shall be rejected. |
| NFR-SEC-05 | Must / Approved | Client errors shall use the documented shape and expose no stack trace, database detail, secret, session identifier or credential. |
| NFR-SEC-06 | Must / Selected implementation baseline | Every mutation, including login/registration, shall require exact configured Origin, JSON content type and X-LabFlow-Request header, and reject cross-site Fetch Metadata. GET shall not change domain state. Untrusted-origin and missing-header tests shall fail. |
| NFR-SEC-07 | Must / Selected implementation baseline | Login/registration shall share a MongoDB-atomic limit of ten attempts per source IP per fixed 15-minute window; the eleventh attempt returns 429 with Retry-After. HMAC window IDs avoid storing raw IPs. Proxy trust requires deployment verification. |
| NFR-SEC-08 | Must / Approved | Secrets shall exist only in runtime configuration. Repository/history and browser artifact scans shall find none; .env.example contains names only. |
| NFR-PRI-01 | Must / Approved | The demo shall use synthetic identities. Stored personal data shall be limited to approved identity fields; retention/deletion policy is TBD before any real use. |

## 5.2 Accessibility

| ID | Priority / status | Requirement and measurable verification |
|---|---|---|
| NFR-ACC-01 | Must / Approved accessibility baseline; draft target | Core screens shall target WCAG 2.2 Level AA and have zero serious or critical automated accessibility findings at the core-flow gate. |
| NFR-ACC-02 | Must / Approved | Registration, login, request, decision, issue, return and damage flows shall complete by keyboard with visible focus and no keyboard trap. |
| NFR-ACC-03 | Must / Approved | Core screens shall remain operable at 200% zoom and 320 CSS pixels without loss of content or action. |

## 5.3 Performance

| ID | Priority / status | Requirement and measurable verification |
|---|---|---|
| NFR-PERF-01 | Must / Approved | All growing list endpoints shall paginate with a default of 20 and hard maximum of 50 records. |
| NFR-PERF-02 | Should / Selected local baseline | Warm sequential paginated reads/dashboard shall target p95 below 250ms on the documented synthetic local dataset; a login sample targets below 2s. Initial client JavaScript shall remain below 100KiB gzip and CSS below 20KiB uncompressed. Record environment and samples; these are not hosted/load guarantees. See docs/QUALITY.md. |
| NFR-PERF-03 | Must / Approved | User actions shall show immediate pending feedback and prevent duplicate submission while a request is in flight. |

## 5.4 Reliability and data integrity

| ID | Priority / status | Requirement and measurable verification |
|---|---|---|
| NFR-REL-01 | Must / Approved | Approval, cancellation, issue, return and conflicting maintenance/archive operations shall be transaction-safe; injected failures shall leave no partial state. |
| NFR-REL-02 | Must / Approved | Real concurrent approval tests shall produce one successful reservation and conflicts for all losers. |
| NFR-REL-03 | Must / Approved | Invalid transitions and repeat mutations shall be idempotent where specified or return 409 without changing persisted state. |

| ID | Priority / status | Requirement and measurable verification |
|---|---|---|
| NFR-REL-04 | Must / Approved | Asynchronous database failures shall reach central error handling and shall not crash the server process during verification. |
| NFR-REL-05 | Must / Approved | /api/health shall return a minimal success response when ready and 503 when database readiness fails. |

## 5.5 Maintainability and reproducibility

| ID | Priority / status | Requirement and measurable verification |
|---|---|---|
| NFR-MNT-01 | Must / Approved | Frontend and backend responsibilities shall remain separated; routes contain mapping, the API client contains HTTP calls and shared circulation rules have one server implementation. |
| NFR-MNT-02 | Must / Approved | The production build and lint commands, once established in M2, shall complete with zero errors before a milestone is accepted. |
| NFR-MNT-03 | Should / Selected implementation baseline | Auth, ownership and circulation modules shall meet the existing 80% aggregate line and branch coverage target, enforced by npm run test:coverage. Do not lower thresholds, skip tests or remove assertions to pass. Browser/process entry coverage is reported separately. |
| NFR-MNT-04 | Must / Approved | A fresh clone shall run by following README prerequisites, environment names and commands without undocumented manual source edits. |
| NFR-MNT-05 | Must / Approved | Each milestone shall update relevant contracts/evidence and use meaningful version-control history; no test or deployment result is claimed before execution. |

## 5.6 Compatibility and deployment

| ID | Priority / status | Requirement and measurable verification |
|---|---|---|
| NFR-COMP-01 | Must / Provisional browser choice | The core MVP shall work in the current stable Chromium browser used for development and at phone/desktop viewport sizes; the browser choice awaits verification review. |
| NFR-COMP-02 | Should / Deferred | Full current Chromium, Firefox and WebKit verification is deferred until the core MVP works; the final required matrix is TBD in M8. |

| ID | Priority / status | Requirement and measurable verification |
|---|---|---|
| NFR-COMP-03 | Must / Approved | A deployed SPA deep link shall refresh without a host-level 404, and browser requests shall not target localhost. |
| NFR-COMP-04 | Must / Approved | The deployed application shall use HTTPS and a host capable of running the chosen Node/Express model. No paid resource shall be provisioned without approval. |

# 6. Lifecycle model

## 6.1 Equipment states

AVAILABLE, RESERVED, ISSUED, MAINTENANCE, ARCHIVED

## 6.2 Request states

PENDING, APPROVED, REJECTED, CANCELLED, ISSUED, RETURNED

## 6.3 State transitions

| Action | Request transition | Equipment transition | Guard |
|---|---|---|---|
| Submit | New → PENDING | Registered asset remains AVAILABLE; Other has no assigned asset | Student authenticated; valid asset or Other description/input; no duplicate active request |
| Approve | PENDING → APPROVED | AVAILABLE → RESERVED | Admin; valid due date; atomic conditional update |
| Reject | PENDING → REJECTED | No change | Admin; reason required |
| Cancel pending | PENDING → CANCELLED | No change | Owner |
| Cancel approved | APPROVED → CANCELLED | RESERVED → AVAILABLE | Owner or admin; admin reason; atomic update |
| Issue | APPROVED → ISSUED | RESERVED → ISSUED | Admin; records linked; due date valid |
| Return serviceable | ISSUED → RETURNED | ISSUED → AVAILABLE | Admin; condition recorded |
| Return damaged | ISSUED → RETURNED | ISSUED → MAINTENANCE | Admin; inspection note; unrelated pending requests do not block return |

| Action | Request transition | Equipment transition | Guard |
|---|---|---|---|
| Enter maintenance | — | AVAILABLE → MAINTENANCE | Admin; no open request; note required |
| Restore | — | MAINTENANCE → AVAILABLE | Admin; no allocation; pending requests allowed; restoration note |
| Archive | — | AVAILABLE or MAINTENANCE → ARCHIVED | Admin; no open request; history retained |

Pending requests do not reserve an asset. Overdue is derived from ISSUED plus a past dueAt; it is not a stored transition and never releases equipment automatically. Reservation expiry remains manual for the MVP.

**Open-request and maintenance rules:** PENDING, APPROVED and ISSUED are open/active; terminal statuses are REJECTED, CANCELLED and RETURNED. The no-open-request guard applies to manual AVAILABLE → MAINTENANCE and archive, not to mandatory damaged return. Other students' pending requests remain unchanged when an asset is reserved, issued or returned damaged. They cannot be approved while the asset is unavailable. After restoration they may be approved only if their requested date is still future; otherwise the admin explicitly rejects or the owner cancels and creates a new request. There is no automatic rejection or date mutation. Restoration permits these pending requests so they cannot deadlock maintenance recovery.

**Date rules:** requestedDueAt and dueAt are ISO 8601 instants with an explicit offset, normalized to UTC. The UI displays them in Asia/Kolkata; seven days means 168 hours, not seven local calendar midnights. Creation requires requestedDueAt > serverNow. Approval requires requestedDueAt > approvalTime and approvalTime < dueAt <= approvalTime + 168 hours. The admin may change the proposed return instant at approval, and the final dueAt is visible in history. Exact boundary equality at the upper limit is valid; equality with the current/approval instant is not future. Malformed/non-future submitted dates return 400; an existing expired pending request fails approval with 409 and remains pending. Issue requires serverNow < dueAt and matching allocation; expiry never silently changes state or extends a loan. An expired approved reservation can only be cancelled under F7 before a fresh request. Requested date need not be rechecked at issue once a valid approval has established dueAt.

**Return inspection:** every return records SERVICEABLE or DAMAGED. When a report exists, the return transaction resolves it with reviewer, server time, matching result and non-empty inspection note, even for a serviceable return. Damaged returns always require a note. Other pending requests are not modified by inspection or return.

# 7. Data requirements

## 7.1 Entities and relationships

```
User (student) 1 ---- * BorrowingRequest * ---- 0..1 Equipment
User (admin)   1 ---- * lifecycle events / inventory updates
BorrowingRequest 1 --- 0..1 embedded DamageReport
User 1 -------- 0..* Session
```

Shared entities are referenced by identifier. Lifecycle events and damage details belong to one request and are embedded with it. A request stores immutable asset name/tag snapshots when a registered asset is selected or assigned, so archived assets do not erase history. The original unlisted description remains unchanged after assignment.

## 7.2 Essential fields

| Entity | Essential fields |
|---|---|
| User | _id, name, normalized email, password hash excluded by default, role, timestamps |
| Equipment | _id, immutable normalized asset tag, name, category, description, location, state, current request reference, condition note, updated-by reference, version, timestamps |
| Borrowing request | _id, student reference, nullable equipment reference until assignment, original requestedEquipment description for Other, actual asset snapshot after assignment, request key, purpose, requestedDueAt, approved dueAt, approval instant in lifecycle event, status, issue/return times, return condition, inspection note, lifecycle events, optional damage report, timestamps |
| Lifecycle event | Action, actor reference, server timestamp, optional reason |
| Damage report | Description, reporter/time, nullable reviewer/time and inspection result (SERVICEABLE/DAMAGED); reviewer/time/result present together when resolved |
| Session | Session identifier/data, user reference and expiry as required by the selected store |

## 7.3 Constraints and indexes

- Unique normalized user email.
- Unique normalized equipment asset tag.
- Unique (studentId, requestKey) for request replay safety.
- Partial unique (studentId, equipmentId) for open/active requests (PENDING, APPROVED, ISSUED).
- The nullable equipment reference permits at most one unassigned open request per student. Unassigned requests have no allocation; approval requires atomic registered-asset assignment and reservation.
- Partial unique equipmentId for approved/issued allocation.
- Query indexes for request owner/status/date, overdue lookup and equipment search/filter fields.
- Strict enums, timestamps, bounded strings, valid identifiers/dates and allowlisted updates.
- RESERVED equipment references exactly one APPROVED request.
- ISSUED equipment references exactly one ISSUED request.
- AVAILABLE, MAINTENANCE and ARCHIVED equipment have no allocation reference.
- Request and equipment lifecycle updates commit together.
- requestedDueAt and dueAt obey Section 6.3; the approval event preserves the server instant used for the 168-hour bound.

- An archived asset is never eligible for new requests or student discovery; existing request snapshots and admin history remain readable.
- A completed return cannot contain an unresolved damage report. Maintenance may coexist with unrelated pending requests only through the damaged-return flow; restoration retains them.

# 8. API requirements

## 8.1 Conventions

- Base path: /api.
- JSON request/response unless status is 204.
- Success: { "data": ... }; lists also include { "pagination": { "page", "limit", "total" } }.
- Error: { "error": { "code", "message", "fields"? } }.
- Dates: ISO 8601 with explicit offset; persisted in UTC and displayed in Asia/Kolkata.
- Authentication: server-side session cookie for protected routes (proposed mechanism).
- Mutations: trusted-origin and CSRF protection required.
- Common errors: 400, 401, 403, 404, 409, 413, 415, 429, safe 500, and readiness 503.

## 8.2 Endpoint contract

| Method/path | Access | Request | Success | Principal errors |
|---|---|---|---|---|
| POST /api/auth/register | Public | JSON: name, email, password | 201, safe user | 400, 409, 429 |
| POST /api/auth/login | Public | JSON: email, password | 200, safe user plus session cookie | 400, 401, 429 |
| POST /api/auth/logout | Authenticated | No business body | 204 and destroyed session | 401, 403 |
| GET /api/auth/me | Authenticated | — | 200, safe current user | 401 |
| GET /api/equipment | Authenticated | Query: search, category, status, page, limit; student scope always excludes archived assets | 200, role-scoped equipment page | 400, 401 |
| GET /api/equipment/:id | Authenticated | Valid asset ID; archived detail is admin-only | 200, equipment detail | 400, 401, 404 |
| POST /api/equipment | Admin | JSON: asset tag, name, category, description, location, condition note as applicable | 201, equipment | 400, 401, 403, 409 |

| Method/path | Access | Request | Success | Principal errors |
|---|---|---|---|---|
| PATCH /api/equipment/:id | Admin | JSON: expected version, action (EDIT_METADATA, ENTER_MAINTENANCE, RESTORE), fields or mandatory note as specified below | 200, updated equipment with incremented version | 400, 401, 403, 404, 409 |
| DELETE /api/equipment/:id | Admin | Valid asset ID | 204, asset archived | 400, 401, 403, 404, 409 |
| POST /api/requests | Student | JSON: exactly one of available non-archived equipmentId or requestedEquipment description, purpose, future requestedDueAt, stable requestKey | 201 new or 200 identical replay | 400, 401, 403, 404, 409 |
| GET /api/requests | Authenticated | Query: status, page, limit; approved admin filters may include overdue/damage | 200, role-scoped page | 400, 401 |
| GET /api/requests/:id | Owner/ Admin | Valid request ID | 200, request/timeline | 400, 401, 404 |
| POST /api/requests/:id/decision | Admin | JSON: action (APPROVE/REJECT), plus future dueAt within 168 hours and required equipmentId when unassigned, or reason; expired requested date blocks approval | 200, updated request | 400, 401, 403, 404, 409 |
| POST /api/requests/:id/cancellation | Owner/ Admin | JSON: reason when admin | 200, updated request | 400, 401, 403, 404, 409 |
| POST /api/requests/:id/issue | Admin | No business body | 200, issued request | 401, 403, 404, 409 |
| POST /api/requests/:id/return | Admin | JSON: condition (SERVICEABLE/DAMAGED); inspectionNote required for damage or any existing report; resolves report with matching condition | 200, returned request and resolved report if present | 400, 401, 403, 404, 409 |
| POST /api/requests/:id/damage-report | Owner | JSON: description | 201, damage report | 400, 401, 404, 409 |

| Method/path | Access | Request | Success | Principal errors |
|---|---|---|---|---|
| GET /api/dashboard | Admin | — | 200, operational counts/lists | 401, 403 |
| GET /api/health | Public | — | 200 ready or 503 unavailable | 503 |

Exact field lengths and optionality are enforced by the boundary schemas and documented in docs/API_DETAILS.md.

PATCH accepts exactly one action per call. Every action requires a non-negative integer expected version; success atomically increments it and records the admin as updatedBy. EDIT_METADATA requires at least one of name, category, description, location, conditionNote; it cannot modify assetTag, status, allocation, server timestamps or version directly. It does not move or release equipment. Metadata edits to archived assets return 409; admin read/history access remains available.

ENTER_MAINTENANCE accepts only version, action and non-empty note; it requires AVAILABLE, no allocation and no open request. RESTORE has the same input shape and requires MAINTENANCE with no allocation; unrelated pending requests are allowed and retained. Both store note as the current condition/restoration note. Unknown actions, missing notes and mixed metadata/transition payloads return 400; stale versions, invalid current states or failed open-request guards return 409. RESERVED, ISSUED and ARCHIVED cannot be assigned through PATCH. Archive remains the existing DELETE operation, guarded transactionally against all open requests. Damaged return remains the return endpoint, not a manual PATCH.

## 9. Main use cases

| ID | Use case | Normal flow | Exceptional flow |
|---|---|---|---|
| UC-01 | Register and log in | Visitor registers with name/email/password as student, logs in and session identity is restored on reload. | Invalid/duplicate input is rejected; role injection fails; rate limit may return 429. |
| UC-02 | Browse catalog | User searches/filters, opens a result and sees current asset state; admin may read archived assets. | Student discovery always excludes archived assets; archived detail returns 404; invalid filters return 400; empty/error states offer next steps. |
| UC-03 | Request equipment | Student selects equipment/purpose or specifies Other, submits once and sees PENDING in history. | Stale asset, duplicate active request or changed idempotency replay returns 409; unknown network outcome triggers state reload. |

| ID | Use case | Normal flow | Exceptional flow |
|---|---|---|---|
| UC-04 | Decide request | Admin assigns a registered available asset for Other, then approves atomically with a future due date within 168 hours, or rejects with reason. | Expired pending request remains pending and approval returns 409; unavailable asset or competing action conflicts without partial state. |
| UC-05 | Cancel request | Owner cancels pending/approved request; approved cancellation releases reservation. | Wrong owner is not disclosed; issued/already-final request returns 409. |
| UC-06 | Issue equipment | Admin confirms an approved reservation and records physical handover. | Expired/invalid due date or mismatched state returns 409. |
| UC-07 | Report damage | Borrower reports damage while request is issued. | Wrong owner/state or duplicate report is rejected without changing the request. |
| UC-08 | Return equipment | Admin inspects/resolves any report, records condition and note as required, then returns asset to availability or mandatory maintenance. | Missing report inspection or required note blocks completion; unrelated pending requests are retained and do not block damaged return; repeated return/race conflicts or failure rolls back. |
| UC-09 | Maintain inventory | Admin edits metadata or uses a versioned PATCH action/note for maintenance/restoration; DELETE archives eligible assets. | Open requests block manual entry/archive, not restoration with unrelated pending requests; invalid action, missing note, stale version or state mismatch is rejected. |
| UC-10 | Review dashboard | Admin views current counts and action queues. | Student is forbidden; failed data load shows safe retryable error. |

# 10. Validation, authorization, concurrency and privacy

## 10.1 Validation

- Frontend validation improves feedback; server validation is authoritative.
- Validate body, path, query, relevant headers and runtime configuration.
- Reject unknown writable fields, MongoDB operators, invalid IDs/dates/enums and oversized text/bodies.
- Selected implementation limits: asset tag 2–40 uppercase characters (letter/digit followed by letters/digits/dot/underscore/hyphen), immutable; equipment name 2–120, category/location 2–80/2–120; purpose 10–500; descriptions/condition notes at most 2,000; required reasons/reports/inspection notes 1–2,000 non-whitespace characters. Search is bounded literal text up to 80. IDs are 24 hexadecimal characters, request keys UUIDs, page 1–10,000 and limit 1

–50 (default 20). Unknown fields are rejected. docs/API_DETAILS.md defines filter semantics, strict PATCH allowlists and explicit IST input conversion.
- Date validation follows Section 6.3; metadata edits and explicit transition actions use the PATCH allowlist in Section 8.2. note and required inspection notes must contain non-whitespace text.
- Forms show inline associated errors and preserve recoverable user input.

### 10.2 Authorization

- Authentication establishes identity; role and ownership checks authorize each action.
- Student data queries include authenticated ownership in the database predicate.
- Unauthorized cross-user records return 404 to avoid confirming existence.
- Admin-only operations reject authenticated students with 403.
- Public input can never assign a role, owner, lifecycle state, decision actor or server timestamp.

### 10.3 Concurrency and consistency

- MongoDB must support transactions in development/integration verification and deployment.
- Approval, cancellation, issue, return and conflicting inventory operations use conditional expected-state updates in one transaction.
- Transaction operations run sequentially; transient failures may be safely retried by the server transaction mechanism.
- Unique/partial indexes are final duplicate/allocation guards.
- Equipment edits use optimistic version matching.
- Manual maintenance entry/archive guards serialize with new request creation; the mandatory damaged-return transaction instead retains unrelated pending requests. Restoration cannot be blocked solely by those pending requests.
- Tests must cover double request submission, simultaneous approvals, approve-versus-cancel, repeated return, report-versus-return, failure between writes and archive/maintenance-versus-request creation.
- After an indeterminate client timeout, the UI reloads authoritative state before enabling another mutation.

### 10.4 Privacy

- Use synthetic demonstration identities and equipment data.
- Collect only fields approved in this SRS; do not collect data “for later.”
- Never place passwords, hashes, session IDs, secrets or full request bodies in logs/screenshots.
- Password hashes are excluded from normal database query output and every API representation.
- A retention/deletion period is TBD; the system must not be presented for real student use until that policy is approved.

## 11. Requirement-to-verification matrix

This table defines planned evidence, not completed results.

| Requirement | Primary verification | Required evidence |
|---|---|---|
| F1 | API integration and browser tests | Name/email/password-only registration, login/logout/session, duplicate/invalid/role-injection checks |

| Requirement | Primary verification | Required evidence |
|---|---|---|
| F2 | API authorization tests | 401, 403, cross-user 404, ownership query evidence |
| F3 | API and responsive UI tests | Search/filter/pagination, four UI states, student archived exclusion and admin archived access |
| F4 | API/database tests | Unique tag; PATCH action/field/note/version checks; manual open-request guards; restoration with retained pending requests |
| F5 | API/database/ browser tests | Future-date creation, replay after expiry, duplicate-open prevention, archived ineligibility and retained history |
| F6 | Real database race and date-boundary tests | One-winner approval, rollback, expired pending conflict, future-date and exact 168-hour upper boundary |
| F7 | API transaction tests | Pending/approved cancellation, single release and issued conflict |
| F8 | API transaction tests | Expired issue blocked; both return conditions; mandatory maintenance with unrelated pending requests; failed/repeated return |
| F9 | Ownership and race tests | Owner/state/duplicate checks; report resolution on damaged and serviceable return; mandatory notes; report-versus-return outcome |
| F10 | API/UI fixture test | Dashboard values matched to known fixture counts |
| F11 | Keyboard, viewport and automated accessibility checks | Core-flow record at 320px/desktop/200% zoom; finding report |
| NFR-SEC-* | Negative API tests and configuration/artifact review | Auth, origin/CSRF, safe errors, rate limit and redacted secret scan |
| NFR-PRI-01 | Data-field and artifact review | Synthetic name/email data only; no credentials or session values in logs/screenshots |
| NFR-ACC-* | Accessibility automation plus manual operation | Zero serious/critical findings and keyboard/zoom checklist |
| NFR-PERF-* | Pagination tests; baseline measurement in M8 | Page-size boundaries and later approved performance record |

| Requirement | Primary verification | Required evidence |
|---|---|---|
| NFR-REL-* | Real replica-set integration and fault tests | Race, rollback, transition and readiness results |
| NFR-MNT-* | Lint/build/coverage and fresh-clone review | Command output, approved coverage threshold and README walkthrough |
| NFR-COMP-* | Browser/deployment smoke tests | Chromium core-flow evidence; later matrix; HTTPS/deep-link checks |

# 12. Assumptions, exclusions and unresolved decisions

## 12.1 Approved assumptions

- Single laboratory scope with two roles: STUDENT, ADMIN.
- Individually tagged assets; one asset per request, assigned at approval for an unlisted need.
- Pending requests do not reserve; admin approval reserves.
- Transactional inventory safety, server RBAC, ownership and boundary validation are mandatory.
- Original ICTAK references remain unchanged and outside Git.
- No paid service without explicit approval.
- Demo defaults: JavaScript/JSX; name/email/password-only registration; maximum seven days (168 hours) from approval; UTC persistence and Asia/Kolkata display.
- Existing cancellation policy is retained: students cancel own pending/approved-unissued requests; admins cancel approved-unissued reservations with a reason.
- No automatic expiry/rejection worker; pending requests and expired reservations require the explicit handling in Section 6.3.

## 12.2 Selected implementation choices

- MongoDB-backed sessions, single origin, identity parameters in docs/SECURITY.md, and bounded fields/filter semantics in docs/API_DETAILS.md.
- Existing 80% critical coverage target and measured local performance budget in docs/QUALITY.md; no threshold was reduced to pass.
- WCAG 2.2 AA remains a target. Automated findings, manual keyboard/reflow, native browser zoom and browser-engine scope must be reported distinctly. The current in-app-browser evidence does not establish full conformance.
- docs/DESIGN.md contains desktop/mobile layouts, navigation, lifecycle and data diagrams.

## 12.3 Exclusions

- Multiple labs/departments and approval hierarchies
- Quantity/basket requests and partial returns
- SSO, email verification, password recovery and notifications
- Booking calendars, renewals and automatic allocation
- QR scanning, images/uploads, fines, payments and repair procurement
- Advanced analytics, exports and real-time updates

- Advanced performance/load, backup restoration and full cross-browser gates before the core MVP works

### 12.4 Operational and assessment limitations

| Limitation | Required action |
|---|---|
| Hobby/Free capacity limits and a recorded transient readiness failure | Do not claim production availability or hosted load guarantees. |
| Native zoom, screen readers and a full browser-engine matrix are unverified | Record these as limitations rather than completed checks. |
| Administrator demonstration access is private | Deliver reviewer credentials through the assessment channel. |
| Retention/deletion policy for real personal data is undefined | Use synthetic data only; define and approve the policy before real-data use. |

The specification covers 19 endpoints. Requirements, measured verification results and operational limitations are distinct; implementation does not establish formal academic acceptance.

## 13. References

1. MERN_Stack_Project_Guidelines_Student_Reference.docx, ICT Academy of Kerala. Authoritative project guidance for project definition, MERN separation, API/database design, validation, security, testing, deployment and final submission evidence. It is a reference guide; its “should” statements require justified decisions rather than blind adoption.

2. Summer Internship Report __ React.docx, ICT Academy of Kerala. Authoritative template for the eventual internship report sections; it is not an SRS template.

3. README.md, LabFlow project overview and current status.

4. docs/MASTER_PLAN.md, approved architecture, scope, requirements, lifecycle and milestone baseline.

5. docs/DECISIONS.md, accepted, proposed and unresolved decisions.

6. ICTAK LMS assignment pages supplied on 25 October 2026: separate SRS submission and final submission of the report, public GitHub URL, hosted URL and major application screenshots.

Official implementation references include React, Node.js, Express, MongoDB, Mongoose, OWASP and WCAG documentation. Exact installed versions are pinned in the root workspace lockfile; source links for selected mechanisms are in docs/SECURITY.md and docs/QUALITY.md. Supporting implementation contracts and layouts are in docs/API_DETAILS.md and docs/DESIGN.md. Measured results live under docs/evidence/; requirements and recorded results remain distinct.